```
import os, random, math, json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
from sklearn.utils.class_weight import compute_class_weight
from tensorflow.keras import layers, Model
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

print("TensorFlow:", tf.__version__)
```

```
TensorFlow: 2.19.0
```

```
SEED = 23
MAX_LEN = 128
BATCH_SIZE = 16
EPOCHS = 50
PATIENCE = 8

EMBED_DIM = 64
ENC_UNITS = 128
LATENT_DIM = 96
DROPOUT = 0.3
LR = 0.001

RECON_LOSS_WEIGHT = 0.20
CLASS_LOSS_WEIGHT = 1.00

def set_seed(seed=SEED):
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    tf.random.set_seed(seed)

set_seed(SEED)
```

```
csv_path = tf.keras.utils.get_file(
    "HIV7.csv",
    "https://raw.githubusercontent.com/McahitKutsal/hivcsv/main/HIV7.csv"
)

df = pd.read_csv(csv_path)
df["smiles"] = df["smiles"].astype(str)
df = df[df["smiles"].str.len() < MAX_LEN].reset_index(drop=True)

print(df.head())
print(df["HIV_active"].value_counts(dropna=False))
print("Dataset size:", len(df))
```

```
Downloading data from https://raw.githubusercontent.com/McahitKutsal/hivcsv/main/HIV7.csv
158661/158661 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
   Unnamed: 0                                     smiles activity  \
0           0                             CNC(=O)Nc1ccncc1       CI
1           1                      O=C1NC(O)c2cccc(Cl)c2O1       CI
2           2                       O=S1(=O)C=Cc2ccccc2CC1       CI
3           3    Cc1ccc(OS(=O)(=O)c2ccccc2[N+](=O)[O-])cc1       CM
4           4  CN(c1ccccc1)S(=O)(=O)c1cccc([N+](=O)[O-])c1       CM

   HIV_active
0           0
1           0
2           0
3           1
4           1
HIV_active
0    1412
1    1377
Name: count, dtype: int64
Dataset size: 2789
```

```
def build_char_maps(train_smiles):
    chars = sorted(set("".join(train_smiles.tolist())))
    # 0: PAD, 1: START, 2: END, 3: UNK
    char_to_idx = {"<PAD>":0, "<START>":1, "<END>":2, "<UNK>":3}
    for ch in chars:
        if ch not in char_to_idx:
```

```
                char_to_idx[ch] = len(char_to_idx)
        idx_to_char = {v:k for k,v in char_to_idx.items()}
        return char_to_idx, idx_to_char

    def encode_smiles_list(smiles_list, char_to_idx, max_len=MAX_LEN):
        x = np.zeros((len(smiles_list), max_len), dtype=np.int32)
        for i, smi in enumerate(smiles_list):
            tokens = [char_to_idx["<START>"]]
            for ch in smi:
                tokens.append(char_to_idx.get(ch, char_to_idx["<UNK>"]))
            tokens.append(char_to_idx["<END>"])
            tokens = tokens[:max_len]
            x[i, :len(tokens)] = tokens
        return x

    def summarize_history(history):
        hist = history.history
        return {
            "loss": hist.get("loss", []),
            "val_loss": hist.get("val_loss", []),
            "accuracy": hist.get("class_output_accuracy", hist.get("accuracy", [])),
            "val_accuracy": hist.get("val_class_output_accuracy", hist.get("val_accuracy", [])),
            "auc": hist.get("class_output_auc", []),
            "val_auc": hist.get("val_class_output_auc", hist.get("val_auc", [])),
        }

    def smooth_curve(values, window=5):
        if len(values) == 0:
            return values
        out = []
        for i in range(len(values)):
            s = max(0, i-window+1)
            out.append(float(np.mean(values[s:i+1])))
        return out

    def pad_hist(histories, key):
        max_len_hist = max(len(h.get(key, [])) for h in histories if len(h.get(key, [])) > 0)
        arr = []
        for h in histories:
            vals = h.get(key, [])
            if len(vals) == 0:
                vals = [0.0] * max_len_hist
            elif len(vals) < max_len_hist:
                vals = vals + [vals[-1]] * (max_len_hist - len(vals))
            arr.append(vals)
        return np.array(arr)

    def find_best_threshold_by_f1(y_true, y_prob):
        thresholds = np.arange(0.10, 0.91, 0.02)
        best_thr, best_f1 = 0.5, -1.0
        for thr in thresholds:
            y_pred = (y_prob >= thr).astype(int)
            score = f1_score(y_true, y_pred, zero_division=0)
            if score > best_f1:
                best_f1 = score
                best_thr = thr
        return float(best_thr), float(best_f1)
```

```
    def make_autoencoder_lstm_model(vocab_size, max_len=MAX_LEN, embed_dim=EMBED_DIM,
                                    enc_units=ENC_UNITS, latent_dim=LATENT_DIM,
                                    dropout=DROPOUT, lr=LR, w0=1.0, w1=1.0):

        encoder_inputs = layers.Input(shape=(max_len,), name="encoder_input")
        x = layers.Embedding(vocab_size, embed_dim, mask_zero=True, name="char_embedding")(encoder_inputs)

        # Encoder
        enc_out = layers.Bidirectional(
            layers.LSTM(enc_units, return_sequences=False, dropout=dropout),
            name="encoder_bilstm"
        )(x)
        latent = layers.Dense(latent_dim, activation="relu", name="latent_dense")(enc_out)
        latent = layers.Dropout(dropout, name="latent_dropout")(latent)

        # Classification head
        cls = layers.Dense(64, activation="relu", name="cls_dense1")(latent)
        cls = layers.Dropout(dropout, name="cls_dropout")(cls)
        class_output = layers.Dense(1, activation="sigmoid", name="class_output")(cls)

        # Decoder / reconstruction head
        dec = layers.RepeatVector(max_len, name="repeat_vector")(latent)
        dec = layers.LSTM(enc_units, return_sequences=True, dropout=dropout, name="decoder_lstm")(dec)
        dec = layers.TimeDistributed(layers.Dense(vocab_size, activation="softmax"), name="recon_output")(dec)

        model = Model(encoder_inputs, [dec, class_output], name="AutoencoderLSTM_HIV")

        def weighted_bce(y_true, y_pred):
            y_true = tf.cast(y_true, tf.float32)
            y_pred = tf.cast(y_pred, tf.float32)
            eps = tf.keras.backend.epsilon()
```

```
        y_pred = tf.clip_by_value(y_pred, eps, 1.0 - eps)
        loss = -(w1 * y_true * tf.math.log(y_pred) + w0 * (1.0 - y_true) * tf.math.log(1.0 - y_pred))
        return tf.reduce_mean(loss)

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=lr),
        loss={
            "recon_output": "sparse_categorical_crossentropy",
            "class_output": weighted_bce
        },
        loss_weights={
            "recon_output": RECON_LOSS_WEIGHT,
            "class_output": CLASS_LOSS_WEIGHT
        },
        metrics={
            "class_output": [
                tf.keras.metrics.BinaryAccuracy(name="accuracy"),
                tf.keras.metrics.AUC(name="auc")
            ]
        }
    )
    return model
```

```
def run_fold(train_df, val_df, test_df, fold_id):
    set_seed(SEED + fold_id)

    char_to_idx, idx_to_char = build_char_maps(train_df["smiles"])
    vocab_size = len(char_to_idx)

    X_train = encode_smiles_list(train_df["smiles"], char_to_idx, MAX_LEN)
    X_val   = encode_smiles_list(val_df["smiles"], char_to_idx, MAX_LEN)
    X_test  = encode_smiles_list(test_df["smiles"], char_to_idx, MAX_LEN)

    y_train = train_df["HIV_active"].astype(np.float32).values
    y_val   = val_df["HIV_active"].astype(np.float32).values
    y_test  = test_df["HIV_active"].astype(np.float32).values

    # Reconstruction target is integer sequence itself
    y_recon_train = X_train.copy()
    y_recon_val   = X_val.copy()

    classes = np.unique(y_train.astype(int))
    weights = compute_class_weight(class_weight="balanced", classes=classes, y=y_train.astype(int))
    class_weight = {int(c): float(w) for c, w in zip(classes, weights)}
    w0 = class_weight.get(0, 1.0)
    w1 = class_weight.get(1, 1.0)

    model = make_autoencoder_lstm_model(vocab_size=vocab_size, w0=w0, w1=w1)

    early_stop = EarlyStopping(
        monitor="val_class_output_auc",
        mode="max",
        patience=PATIENCE,
        restore_best_weights=True,
        verbose=1
    )
    reduce_lr = ReduceLROnPlateau(
        monitor="val_class_output_loss",
        mode="min",
        factor=0.5,
        patience=3,
        min_lr=1e-6,
        verbose=1
    )

    history = model.fit(
        X_train,
        {"recon_output": y_recon_train, "class_output": y_train},
        validation_data=(X_val, {"recon_output": y_recon_val, "class_output": y_val}),
        epochs=EPOCHS,
        batch_size=BATCH_SIZE,
        callbacks=[early_stop, reduce_lr],
        verbose=0,
        shuffle=True
    )

    # Validation threshold tuning
    val_recon, val_probs = model.predict(X_val, verbose=0)
    val_probs = val_probs.reshape(-1)
    best_threshold, best_val_f1 = find_best_threshold_by_f1(y_val, val_probs)

    # Test
    test_recon, test_probs = model.predict(X_test, verbose=0)
    test_probs = test_probs.reshape(-1)
    y_pred = (test_probs >= best_threshold).astype(int)

    fold_result = {
        "fold": fold_id,
        "test_accuracy": accuracy_score(y_test, y_pred),
```

```
        "test_precision": precision_score(y_test, y_pred, zero_division=0),
        "test_recall": recall_score(y_test, y_pred, zero_division=0),
        "test_f1": f1_score(y_test, y_pred, zero_division=0),
        "test_roc_auc": roc_auc_score(y_test, test_probs),
        "best_threshold": float(best_threshold),
        "val_best_f1": float(best_val_f1),
        "epochs_ran": len(history.history["loss"])
    }

    return fold_result, summarize_history(history)
```

```
all_fold_results = []
all_histories = []

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
X_index = np.arange(len(df))
y = df["HIV_active"].values

for fold_id, (train_idx, temp_idx) in enumerate(skf.split(X_index, y), start=1):
    train_df = df.iloc[train_idx].reset_index(drop=True)
    temp_df = df.iloc[temp_idx].reset_index(drop=True)

    val_idx, test_idx = train_test_split(
        np.arange(len(temp_df)),
        test_size=0.5,
        random_state=SEED + fold_id,
        stratify=temp_df["HIV_active"]
    )

    val_df = temp_df.iloc[val_idx].reset_index(drop=True)
    test_df = temp_df.iloc[test_idx].reset_index(drop=True)

    print(f"\n===== Fold {fold_id} =====")
    print("Train:", len(train_df), "Val:", len(val_df), "Test:", len(test_df))

    fold_result, hist = run_fold(train_df, val_df, test_df, fold_id)
    print(fold_result)
    all_fold_results.append(fold_result)
    all_histories.append(hist)

results_df = pd.DataFrame(all_fold_results)
results_df
```

```
===== Fold 1 =====
Train: 2231 Val: 279 Test: 279

Epoch 19: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.

Epoch 22: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.

Epoch 29: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.

Epoch 32: ReduceLROnPlateau reducing learning rate to 3.125000148429535e-05.

Epoch 35: ReduceLROnPlateau reducing learning rate to 1.5625000742147677e-05.

Epoch 38: ReduceLROnPlateau reducing learning rate to 7.812500371073838e-06.
Epoch 38: early stopping
Restoring model weights from the end of the best epoch: 30.
{'fold': 1, 'test_accuracy': 0.9068100358422939, 'test_precision': 0.9826086956521739, 'test_recall': 0.824817518

===== Fold 2 =====
Train: 2231 Val: 279 Test: 279

Epoch 14: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.

Epoch 20: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.

Epoch 23: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.

Epoch 26: ReduceLROnPlateau reducing learning rate to 3.125000148429535e-05.

Epoch 29: ReduceLROnPlateau reducing learning rate to 1.5625000742147677e-05.
Epoch 30: early stopping
Restoring model weights from the end of the best epoch: 22.
{'fold': 2, 'test_accuracy': 0.956989247311828, 'test_precision': 1.0, 'test_recall': 0.9124087591240876, 'test_f

===== Fold 3 =====
Train: 2231 Val: 279 Test: 279

Epoch 6: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.

Epoch 9: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.

Epoch 12: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.
Epoch 14: early stopping
Restoring model weights from the end of the best epoch: 6.
{'fold': 3, 'test_accuracy': 0.931899641577061, 'test_precision': 0.976, 'test_recall': 0.8840579710144928, 'test

===== Fold 4 =====
Train: 2231 Val: 279 Test: 279
Epoch 12: early stopping
Restoring model weights from the end of the best epoch: 4.
{'fold': 4, 'test_accuracy': 0.9175627240143369, 'test_precision': 0.9389312977099237, 'test_recall': 0.891304347

===== Fold 5 =====
Train: 2232 Val: 278 Test: 279

Epoch 8: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.

Epoch 19: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.

Epoch 22: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.

Epoch 31: ReduceLROnPlateau reducing learning rate to 3.125000148429535e-05.

Epoch 45: ReduceLROnPlateau reducing learning rate to 1.5625000742147677e-05.
Epoch 46: early stopping
Restoring model weights from the end of the best epoch: 38.
{'fold': 5, 'test_accuracy': 0.9175627240143369, 'test_precision': 0.9831932773109243, 'test_recall': 0.847826086
```

	fold	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc	best_threshold	val_best_f1	epochs_ran
0	1	0.906810	0.982609	0.824818	0.896825	0.962064	0.60	0.922481	38
1	2	0.956989	1.000000	0.912409	0.954198	0.970135	0.66	0.914062	30
2	3	0.931900	0.976000	0.884058	0.927757	0.949892	0.28	0.905660	14
3	4	0.917563	0.938931	0.891304	0.914498	0.962894	0.48	0.927757	12
4	5	0.917563	0.983193	0.847826	0.910506	0.945781	0.58	0.925490	46

```
summary_rows = []
metric_map = {
    "ACCURACY": "test_accuracy",
    "PRECISION": "test_precision",
    "RECALL": "test_recall",
    "F1": "test_f1",
```

```
    "F1": "test_f1",
    "ROC_AUC": "test_roc_auc",
}
for nice, col in metric_map.items():
    mean = results_df[col].mean()
    std = results_df[col].std(ddof=1)
    var = results_df[col].var(ddof=1)
    summary_rows.append({
        "Metric": nice,
        "Mean": mean,
        "Std": std,
        "Variance": var,
        "Formatted": f"{mean:.3f} ± {std:.3f}"
    })

summary_df = pd.DataFrame(summary_rows)
summary_df
```

	Metric	Mean	Std	Variance	Formatted
0	ACCURACY	0.926165	0.019401	0.000376	0.926 ± 0.019
1	PRECISION	0.976147	0.022613	0.000511	0.976 ± 0.023
2	RECALL	0.872083	0.035219	0.001240	0.872 ± 0.035
3	F1	0.920757	0.021704	0.000471	0.921 ± 0.022
4	ROC_AUC	0.958153	0.010033	0.000101	0.958 ± 0.010

```
final_table = pd.DataFrame([{
    "Model": "Autoencoder-LSTM",
    "Accuracy": summary_df.loc[summary_df["Metric"]=="ACCURACY", "Formatted"].values[0],
    "Precision": summary_df.loc[summary_df["Metric"]=="PRECISION", "Formatted"].values[0],
    "Recall": summary_df.loc[summary_df["Metric"]=="RECALL", "Formatted"].values[0],
    "F1": summary_df.loc[summary_df["Metric"]=="F1", "Formatted"].values[0],
    "ROC-AUC": summary_df.loc[summary_df["Metric"]=="ROC_AUC", "Formatted"].values[0],
}])

print(final_table.to_string(index=False))
```

```
           Model      Accuracy     Precision        Recall            F1       ROC-AUC
Autoencoder-LSTM 0.926 ± 0.019 0.976 ± 0.023 0.872 ± 0.035 0.921 ± 0.022 0.958 ± 0.010
```

```
results_df.to_csv("AutoencoderLSTM_fold_results.csv", index=False)
summary_df.to_csv("AutoencoderLSTM_summary_results.csv", index=False)
final_table.to_csv("AutoencoderLSTM_final_table.csv", index=False)
print("Saved:")
print("- AutoencoderLSTM_fold_results.csv")
print("- AutoencoderLSTM_summary_results.csv")
print("- AutoencoderLSTM_final_table.csv")
```

```
Saved:
- AutoencoderLSTM_fold_results.csv
- AutoencoderLSTM_summary_results.csv
- AutoencoderLSTM_final_table.csv
```

```
# Average curves across 5 folds
loss_arr = pad_hist(all_histories, "loss")
val_loss_arr = pad_hist(all_histories, "val_loss")
acc_arr = pad_hist(all_histories, "accuracy")
val_acc_arr = pad_hist(all_histories, "val_accuracy")
auc_arr = pad_hist(all_histories, "auc")
val_auc_arr = pad_hist(all_histories, "val_auc")

plt.figure(figsize=(7,4))
plt.plot(smooth_curve(loss_arr.mean(axis=0), window=5), label="Train Loss")
plt.plot(smooth_curve(val_loss_arr.mean(axis=0), window=5), label="Val Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Autoencoder-LSTM Average 5-Fold Loss")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(7,4))
plt.plot(smooth_curve(acc_arr.mean(axis=0), window=5), label="Train Accuracy")
plt.plot(smooth_curve(val_acc_arr.mean(axis=0), window=5), label="Val Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Autoencoder-LSTM Average 5-Fold Accuracy")
plt.legend()
plt.grid(True)
plt.show()
```

```
plt.figure(figsize=(7,4))
plt.plot(smooth_curve(auc_arr.mean(axis=0), window=5), label="Train AUC")
plt.plot(smooth_curve(val_auc_arr.mean(axis=0), window=5), label="Val AUC")
plt.xlabel("Epoch")
plt.ylabel("AUC")
plt.title("Autoencoder-LSTM Average 5-Fold AUC")
plt.legend()
plt.grid(True)
plt.show()
```



